



ONLINE JUDGE · COURSE PROJECT

程序设计训练在线评测系统

OJ Python Homework 2 实验报告

用 FastAPI、Streamlit 与 SQLite 完成题库、判题、权限、日志、AI 命题和学习分析的一体化课程实验。

课程	程序设计训练 (Python)
学生	王健成 · s10610049
日期	2026 年 9 月 10 日

从一次提交，到一条可追溯的学习路径

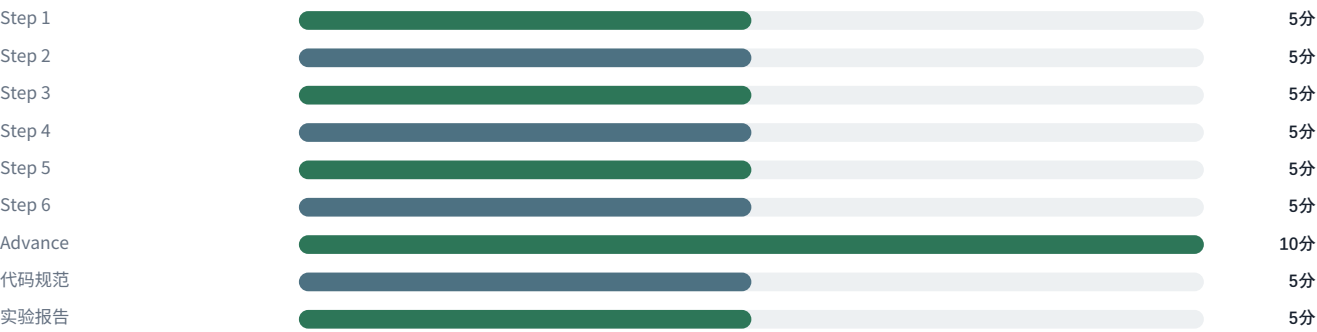
02 / 实验目标与要求对照

我想解决的问题

这次实验并不是简单地做一个“运行代码的网页”。我把它理解为一个完整链路：教师能够维护题目，学生能够登录、做题和查看结果，系统能够在资源边界内稳定判题，管理员能够追溯日志和观察整体学习表现，AI 功能则必须真正接入命题流程，而不是孤立的聊天框。

课程要求的主线是 Step 1 至 Step 6，所有业务 API 使用 FastAPI 的 `async def`，前端必须使用 Streamlit 并通过 REST API 访问后端。Advance 要求 AI 命题具备交互、模型配置、实时进度、终止以及 Token 和价格统计。本项目在保留这些合同的基础上增加了双语、题目包导入、学习分析、持久化修订和编程助手。

课程评分结构（总分 50）



要求组	项目中的落点	验收重点
Step 1-6	题目、判题、提交、用户、日志、Streamlit	路径、权限、状态码与真实交互
Advance	异步 AI 命题会话和多轮修订	真实进度、停止、用量、入库前审查
用户追加	双语、导入、分析、助手、Demo 数据	功能完整性、隐私和前端一致性

本报告的证据边界

自动测试、浏览器验收、真实模型调用、Linux 判题和 GitHub CI 是不同证据。后文分别列出，不用其中一种代替另一种。

03 / 系统架构与技术选型

四层结构

我把系统拆成界面层、API 层、领域服务层和运行数据层。这样做最直接的收益是：Streamlit 不会绕过权限直接访问数据库；判题和 AI 任务可以独立维护生命周期；同一套统计口径可以同时服务学生端、管理端和编程助手。

一次用户操作的主链路



层次	主要技术	选择原因
前端	Streamlit + 本地字体 + CSS	满足课程约束，同时保持中文、代码和长题目的可读性
API	FastAPI + Uvicorn	异步路由、依赖注入和统一错误处理清楚
存储	SQLite 文档命名空间	本地部署简单，事务与热备份足以支撑课程 Demo
执行	asyncio + 独立进程 + psutil	不阻塞 API，并能回收编译器和程序的后代进程
AI	DeepSeek 兼容 SSE 接口	可以边接收边更新真实进度，并统计提供商用量

```
@application.post("/api/submissions/")
async def submit(request: Request, user=Depends(current_user)):
    value = await body_object(request)
    # 校验频率、题目、语言并保存 pending 快照
    launch(submission, problem, language)
    return response({"submission_id": submission["submission_id"],
                    "status": "pending"})
```

这段入口体现了本项目的基本约束：先认证与校验，再把可恢复的 `pending` 状态落库，最后由后台任务执行耗时工作。浏览器不需要等待全部测试点跑完才能收到提交编号。

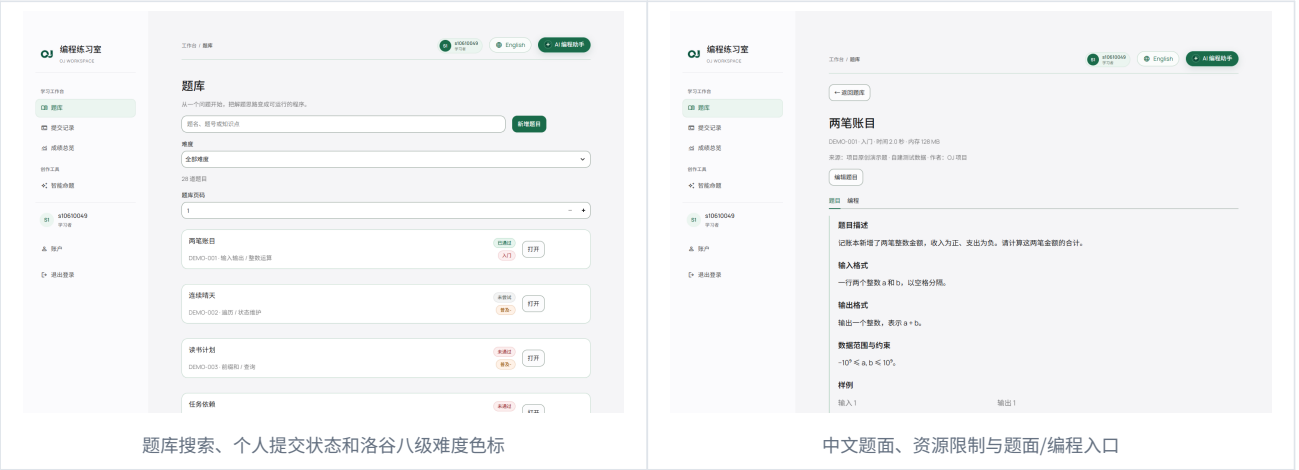
04 / Step 1：题目生命周期、双语与导入

不只是增删改查

题目使用稳定 `problem_id`，包含题面、输入输出说明、样例、测试点、参考解、标签、难度和资源限制。创建和更新前统一校验字段；更新会改变题目版本摘要，历史提交因此可以标记为旧版本。题目详情按课程合同向已登录用户返回 `testcases`，判题器内部参考解和 AI 校验草稿则不作为普通题目字段回传。

双语没有采用“切换按钮后临时机翻”的做法。英文题面作为带版本关系的 `translations.en` 保存，中文题面变化后旧译文会标记为过期；英文界面显示英文标题、说明和元数据，技术诊断则保留必要的原文边界。

题库与题面（最终网站截图）



扩展能力	实现方式	安全边界
洛谷八级难度	<code>shared/taxonomy.py</code> 维护稳定 ID、中英文标签和颜色令牌	旧标签显式迁移，不向用户显示“旧制”噪声
249 个知识点	<code>shared/knowledge.py</code> 提供可搜索选项，同时允许自定义输入	显示标签与内部 ID 分离
标准 ZIP 导入	上传、解析预览、人工确认、原子提交两阶段	限制总大小、成员数、路径穿越和压缩炸弹
洛谷数据包	识别常见 <code>.in/.out/.ans</code> 配对，缺题面时补元数据	不抓取或分发官方隐藏测试数据
参考附件	文本、代码、表格、文档、PDF、图片和受限 ZIP	只向模型传有界解析快照，不传本机路径

题目包采用 `OJ Problem Archive v1` 清单。用户可以下载模板，预览阶段清楚看到来源格式、字段、样例、测试点和冲突；已有题号只有在明确勾选覆盖并提交版本摘要时才会被替换。

05 / Step 2: 判题、语言与资源限制

从源码到测试点结果

判题器先解析语言配置，再在独立临时目录写入源码。C++ 先编译，Python 通过隔离入口执行；每个测试点独立提供标准输入、捕获标准输出，并用规范化后的文本比较答案。最终结果既有任务级 `pending/success/error`，也有测试点级 `AC`、`WA`、`CE`、`TLE`、`MLE`、`RE` 和基础设施错误。

判题状态机



```
async def run_command(argv, directory, stdin, time_limit, memory_limit, ...):
    worker = asyncio.create_task(asyncio.to_thread(_execute, ...))
    try:
        return await asyncio.shield(worker)
    except asyncio.CancelledError:
        cancel.set()
        # 等待工作线程清理完整进程树后，再释放临时目录
        ...

    resource.setrlimit(resource.RLIMIT_AS, (memory_bytes, memory_bytes))
    resource.setrlimit(resource.RLIMIT_CPU, (cpu_limit, cpu_limit + 1))
```

Windows 开发环境使用墙钟、进程树和内存监控；Linux 代表环境再应用 `RLIMIT_AS`、`RLIMIT_CPU`、`RLIMIT_CORE` 和输出文件限制。这里我刻意把“本机可运行”和“Linux 限制已验证”分开记录，因为课程最终判题以 Linux 兼容为目标。

时间与内存按“题目配置 > 语言配置 > 系统默认 3 秒 / 128 MB”解析。动态语言注册和语言列表接口仍保留，所有已登录用户均可注册语言；服务端会对执行模板、占位符和参数执行严格白名单校验，避免把任意系统命令注册成语言。

06 / Step 3 与 Step 5：提交、复评、日志和审计

可追溯的提交快照

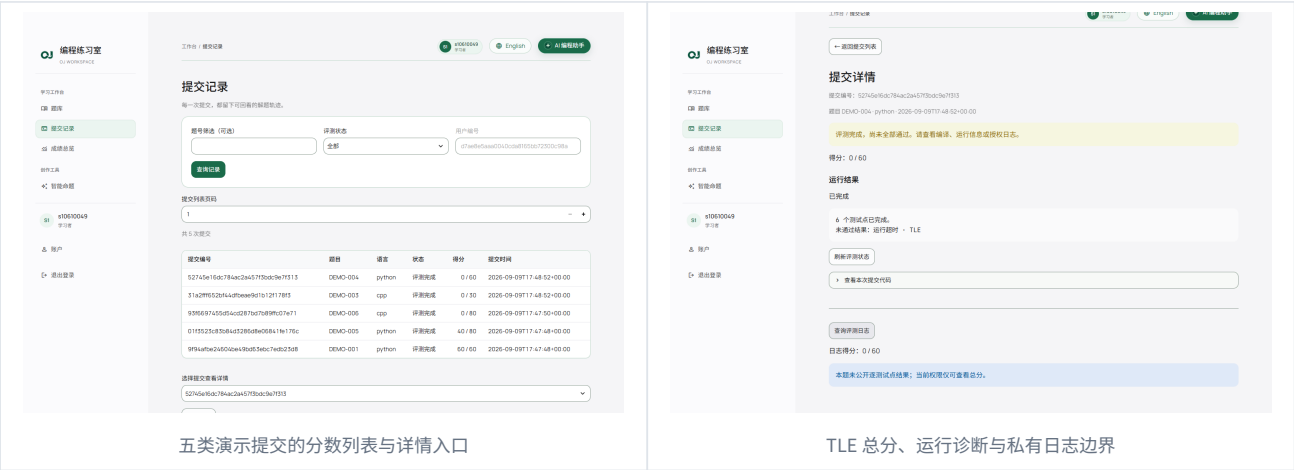
每次提交保存当时的题目标题、难度、标签和题目版本摘要。列表只返回摘要；详情返回总分和编译、运行信息；测试点日志通过独立接口读取。管理员重新评测时会增加修订号并重新执行，原提交编号不变，因此页面能区分“同一次提交的复评”和“新的提交”。

场景	提交者本人	其他登录用户	管理员
提交摘要	可见自己的	默认不可见	按接口权限可见
public_cases=False 日志	仅总分和总数，无测试点明细	403	可按策略检查
public_cases=True 日志	可见完整 details	可见完整 details	可见
他人提交源码/AI 私有上下文	不通过日志接口泄露	不泄露	管理统计也只使用聚合值

```
if user["role"] != "admin":
    records = [s for s in records if s["user_id"] == user["user_id"]]
if user_id:
    records = [s for s in records if s["user_id"] == user_id]
```

日志公开与代码公开是两件不同的事。即使测试点日志可见，系统也不会同时公开提交者源码、AI 提示或题目维护字段。访问日志会留下读取者、提交和时间等审计信息，便于演示权限边界。

提交状态和日志（最终网站截图）



五类演示提交的分数列表与详情入口

TLE 总分、运行诊断与私有日志边界

07 / Step 4：登录、会话与管理员权限

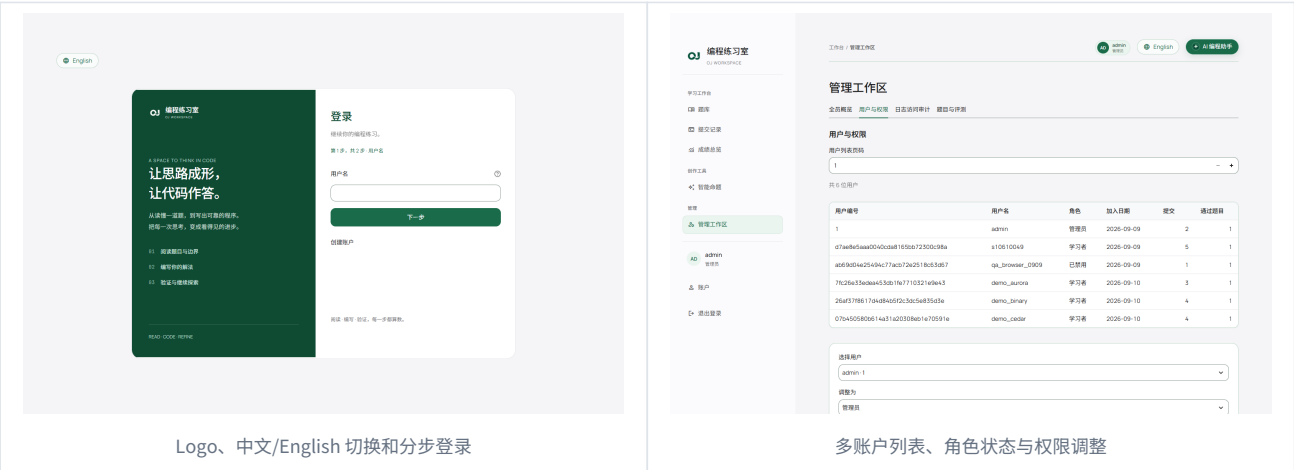
账户不是一行明文密码

注册和登录页面采用分步、克制的表单结构。密码用 bcrypt 哈希保存；登录成功后后端生成随机会话令牌，并通过 HttpOnly、SameSite=Lax Cookie 返回给会话化 HTTP 客户端。Streamlit 为每个前端会话维护独立 cookie jar，不把令牌写入页面状态。未知用户名也会执行同等成本的哈希校验，降低用户名枚举的时间差。

```
hashed = user["password_hash"] if user else fallback_hash
matched = await asyncio.to_thread(
    bcrypt.checkpw, password_bytes(password), hashed.encode()
)
token = secrets.token_urlsafe(32)
result.set_cookie(COOKIE, token, httponly=True,
    samesite="lax", secure=is_https)
```

角色	典型权限	明确禁止
学习者	看题、提交、看自己的记录和分析、使用 AI	管理他人账户、改日志策略、查看他人私有提交
管理员	题目维护、语言、复评、角色、全员统计和审计	API 仍需登录，不能绕过参数和版本校验
禁用账户	保留数据以供审计	登录返回 403

认证与管理员入口（最终网站截图）



课程初始管理员仅用于本地实验。Demo 账号和运行数据由幂等脚本创建，密码从无回显输入或被 Git 忽略的本地配置读取，不写进脚本、报告、截图或提交历史。

08 / Step 6：Streamlit 前端与设计系统

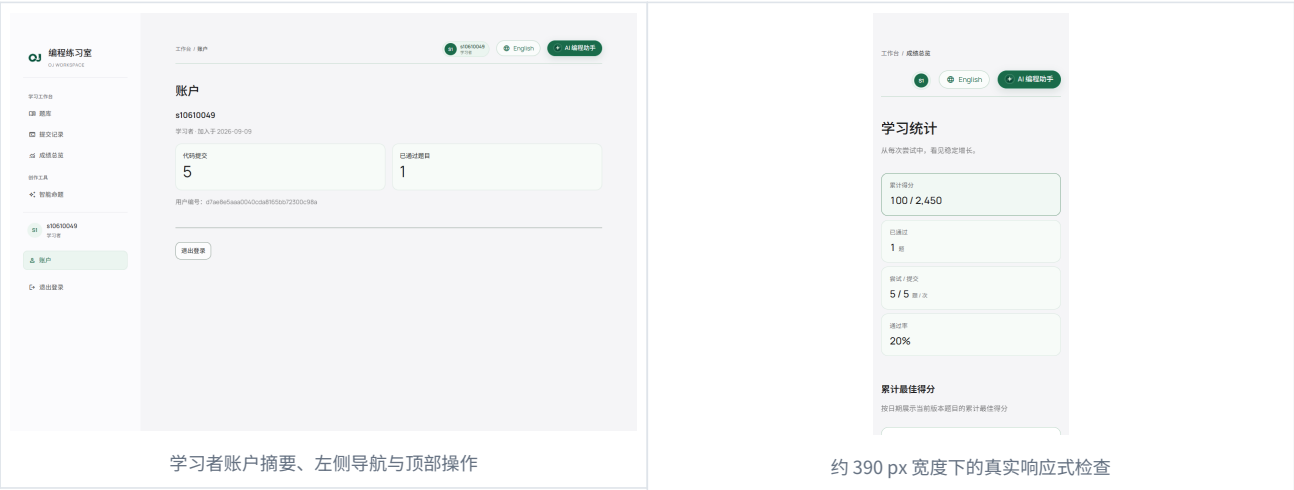
Python 前端也能有一致的产品感

课程要求前端使用 Streamlit，所以我没有另建 React 页面。所有业务交互仍由 `frontend/client.py` 发起 HTTP 请求，Cookie 会话、状态码和统一 `{code, msg, data}` 响应都在客户端集中处理。题库、提交、智能命题、成绩总览、账户和管理工作区只是不同路由，共享同一套导航和设计令牌。

视觉上以灰白背景、深墨文字和低饱和绿色为主。学生端留白更舒适，管理端表格更紧凑；按钮、圆角、边框、状态色和字体全部统一。本地 Manrope、Noto Sans SC 与 JetBrains Mono 字体避免依赖 Google 外网，代码区域和中文题面都能长时间阅读。

体验细节	处理方式
中英文切换	顶部右侧胶囊按钮；固定文案、题面和元数据同步切换
难度与状态	颜色不是唯一信息，标签文字同时说明等级和判题结果
响应式布局	宽屏保留导航和数据密度，窄屏收为单列并保持操作顺序
动效	抽屉、登录衔接、加载状态使用短时缓动；尊重 reduced motion
Streamlit 外壳	隐藏 Deploy 与默认菜单，不暴露框架噪声

学习工作区（最终网站截图）



界面参考 PathHub 的字体、绿色体系和轻量动效，但没有修改参考项目，也没有复制其教育行业系统提示词。Logo 和 AI 头像使用本项目自己的品牌资源。

09 / Advance：异步 AI 智能命题

生成不是一次阻塞请求

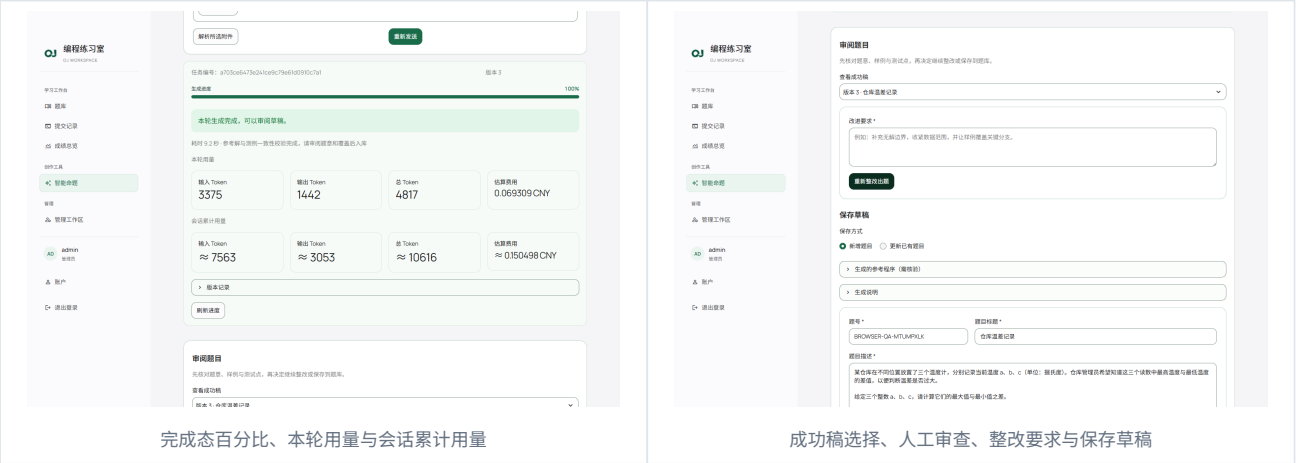
智能命题先让用户选择知识点和洛谷八级目标难度，也允许把自定义要求直接写进 Prompt。创建任务后，模型通过 SSE 流式返回正文；后端按“连接、接收、解析、自动修正、一致性检查”更新真实阶段和数值进度。前端展示百分比、进度条、耗时、停止按钮和编程相关加载文案，不用假的定时动画冒充模型进度。

持久化命题会话



```
progress("正在执行测试数据生成器（第 1 次）")
generated = _generator_inputs(await_execute(generator, "", "generator"))
progress("正在复跑测试数据生成器并检查确定性（第 2 次）")
if generated != repeated:
    fail("generator_nondeterministic")
```

AI 命题过程（最终网站截图）



运行中输入栏默认锁定，点击“编辑要求”后才可修改并重新发送；重发会停止当前任务、继承会话和附件，形成新修订。完成后可以从任一成功历史稿出发整改，避免用户只能接受最新版本。

10 / AI 质量、附件、费用与失败语义

让“每次能生成”建立在可解释的兜底上

模型被要求返回结构化题目、完整英文译文、参考解、字面测试点、确定性测试生成器和生成说明。系统不会因为 JSON 能解析就判定成功，而是经过字段校验、安全静态检查、两次生成器复跑、参考解逐例执行和答案比对。常见的安全主入口会被精确归一化；其他危险导入、文件访问、进程、网络和非确定性行为仍然拒绝。

失败类型	对用户的语义	可恢复动作
输入不足	400，指出命题要求或附件问题	编辑要求后重发
模型输出不完整	在有界次数内自动修正，仍失败则保留失败修订	查看错误，基于原请求再开修订
一致性失败	不通过 <code>authoring_check</code> ，不自动入库	改 Prompt 或人工核对后再生成
模型/网络不可用	区分限流、超时和依赖失败	稍后恢复会话，不丢已完成稿
用户停止	后端实际取消活动任务并记录 <code>stopped</code>	从同一会话重新发送

附件上传有独立大小、成员数、类型和提取长度上限。PDF、DOCX、表格和文本会变成带来源摘要的有限字符快照；图片会先校验格式、像素和帧数，本地默认仅提供元数据，只有显式启用视觉能力时才传递受限图像字节，不冒充 OCR。可疑“泄露密钥/覆盖系统指令”内容按不可信参考资料处理。

Token 统计优先使用提供商返回的输入/输出值，缺字段时才明确标为估算。费用按配置单价和计价单位计算；界面始终给出数值，不显示“未知”，同时说明它是估算而不是最终账单。密钥只在后端运行环境读取，模型配置接口不会回显它。

源稿建立时的命题与题库快照

99 命题会话	117 修订记录	28 / 28 英文题面	245 测试点
------------	-------------	-----------------	------------

11 / 全局编程助手与私有学习上下文

随时可用，但不越过用户边界

右上角 AI 编程助手以抽屉方式展开，进入后先用一段简短介绍说明能力：解释概念、定位错误、提供启发式提示并帮助复盘。回答风格要求专业、精简，优先引导思考；只有用户明确需要时才给完整代码。

真正困难的部分不是聊天 UI，而是“它知道什么”。每次发送消息前，后端重新读取当前用户的学习进度，生成版本化、可验证的上下文快照。内容包括完成数、最高分、难度/知识点聚合、最近活动，以及用户显式选中的题目、代码或诊断。快照不含其他账户数据、密码、会话令牌、隐藏测试点或管理员私有字段。

```
snapshot = build_progress_snapshot(
    problems, submissions, user_id,
    difficulty_normalizer=normalize_difficulty,
    generated_at=now,
)

# 超出预算时按确定顺序截断，而不是随机丢失上下文
context["per_problem"] = original[:kept]
context["coverage"]["status"] = "partial"
```

上下文区块	用途	隐私控制
学习汇总	判断完成度和近期趋势	只从当前 user_id 计算
单题表现	针对薄弱题给提示	只传最高分、尝试和状态摘要
选中的代码/诊断	解释这一次错误	需要用户显式选择，长度受限
题目详情	理解公开题意和样例	public_cases=False 时不含隐藏明细



编程助手的右侧抽屉、能力介绍与输入区

会话和回答任务同样持久化，支持停止、恢复和删除；服务重启时未完成回答会得到明确终态，不会假装仍在生成。

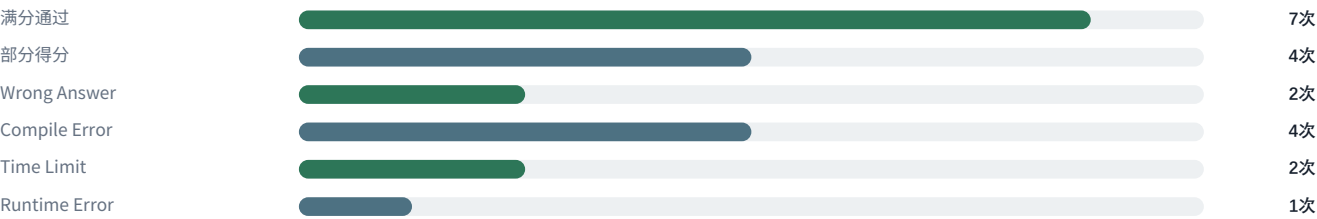
12 / 个人成绩、管理员总览与 Demo 数据

同一统计内核，两种观察尺度

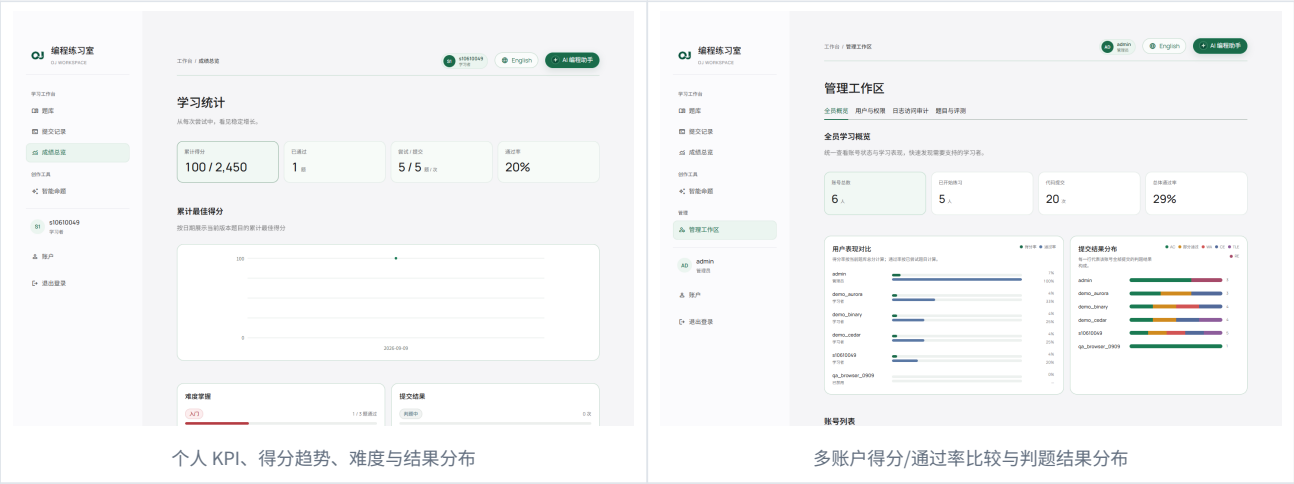
个人成绩页从提交快照计算完成度、最高分、得分趋势、难度分布、知识点掌握和近期题目表现。管理员总览复用完全相同的个人统计函数，再聚合为账户比较，因此“学生看到的分数”和“管理员表格里的分数”不会使用两套口径。

管理员页面只返回账户身份、角色和聚合指标，不返回源码或判题明细。页面支持用户名搜索、角色/状态筛选、表现排序、总体 KPI、得分对比和判题结果分布；空数据、零提交和筛选无结果也有明确状态。

2026-09-10 06:31 本地 Demo 提交构成 (共 20 次)



成绩与管理面板 (最终网站截图)



本地演示数据库快照



Demo 脚本会创建差异化账户，让每个账户同时留下 AC、部分分和 CE，并按画像补充 WA 或 TLE；运行库中另有一条独立 RE 记录用于总览。命题记录包含首稿和整改稿。脚本先做 SQLite 热备份，操作幂等，重复运行不会无限复制同一痕迹。

13 / 验证方法、边界测试与结果

不是只看“能打开首页”

我把验证分为合同、领域逻辑、真实集成、浏览器、平台和发布六层。单元测试适合快速定位，FastAPI/HTTP 集成测试验证权限和状态码，Streamlit AppTest 验证表单与状态，浏览器检查真实像素和动效，真实模型矩阵验证提供商路径，Linux/CI 则补本机无法代表的资源限制。

门禁	最终证据
完整 Pytest	同一候选: A0 972 passed / 6 skipped / 138.91 s; 独立 A5 969 / 9 / 133.11 s; 均 978 collected、0 failed
Black / Flake8	Black、Flake8、diff check、66 包依赖检查全部通过
桌面与约 390 px 浏览器	96 项唯一真实交互通过、0 浏览器错误; 桌面与 390 px, 13 张截图
Linux 判题与进程限制	固定 4 项 POSIX 专属; Windows 策略间歇跳过 2/5 项; 最终同 SHA Ubuntu CI 补证
19 组真实 AI 命题矩阵	19/19 完成; 独立 oracle 与典型错解门通过
GitHub Actions	外部门: 最终 push 后以同 SHA Actions 为准
发布提交	main; 精确 SHA 见仓库提交记录

重点反例包括：未登录 401、权限不足 403、非法字段 400、提交限频 429、版本冲突 409、资源不存在 404；Python/C++ 的 AC、WA、CE、TLE、MLE、RE；私有日志不泄露 details；ZIP 路径穿越和压缩炸弹；附件提示注入；AI 取消后任务真正停止；服务重启后活动任务进入可解释终态；英文界面不混入中文业务文案。

平台边界

Windows 上的完整测试不能证明 Linux rlimit 已生效。最终结论必须引用真实 Linux 或 GitHub CI 证据；若仍未获得，应明确写“未验证”，不能改成“通过”。

截至源稿建立时，本地演示库含 28 道题、245 个测试点、6 个账户和 20 次完成提交。数据库快照只是 Demo 数据证据，不替代代码测试；真实模型生成成功也不等于每道题在数学上已被证明正确，最终仍需要人工审题。

14 / 工程过程、时间与 AI 使用

时间投入说明

开发发生在 2026 年 9 月 9 日至 10 日。下面是我根据任务规模回溯的“团队等效工作量”区间，包含并行的自动测试、模型等待和独立审查，不是计时器导出的个人净工时，也不应相加后解释为连续在线时长。

阶段	估算工作量	主要内容
需求与架构	3-4 小时	课程文档、助教答疑、合同和模块边界
API、存储与权限	5-6 小时	Step 1、3、4、5 与错误优先级
判题与资源控制	5-6 小时	Python/C++、进程树、限制和结果映射
Streamlit 与双语体验	6-8 小时	学生端、管理端、响应式和动效
AI 命题与编程助手	9-11 小时	流式任务、取消、修订、附件、上下文
分析、Demo、测试和文档	9-12 小时	图表、演示痕迹、回归、指南和报告

人与 AI 的分工

我把“人 7、AI 3”理解为责任权重，而不是伪造的代码行或 Token 遥测。需求取舍、权限含义、架构边界、验收标准、失败处理和最终责任由人主导；AI 用于候选实现、重复性测试、文案初稿和缺陷搜索。AI 生成的代码仍要经过测试、静态检查和人工审阅，模型建议与最终实现不一致时，以课程合同和可复核证据为准。

责任权重（非遥测）



最终责任没有外包

AI 可以加快候选实现和缺陷搜索，但需求裁决、权限边界、发布判断、报告事实和课程提交责任仍由我承担。

15 / 实验体会、局限与下一步

实验体会

最开始我以为最难的是把 Python 和 C++ 跑起来，真正做下去以后才发现，难点其实是让同一个状态在很多地方保持一致。比如题目被修改后，旧提交怎样解释；日志公开以后，哪些信息仍然不能泄露；AI 生成到一半时修改要求，旧任务怎样真正停止；英文界面切换后，题目、标签和错误提示怎样不各说各话。这些问题单看一个函数都不复杂，但跨过 API、数据库和页面以后就容易出现缝隙。

这次我最大的收获是学会先写合同，再写界面。统一响应结构、版本摘要、任务状态和统计快照看起来增加了前期工作，后面却让我能更有把握地增加成绩面板、历史稿整改和编程助手，而不用推翻基础模块。判题部分也让我更直观地理解了异步：`async def` 不是把所有事情都塞进事件循环，而是让 API 调度、线程工作和外部进程各自待在合适的位置，并在取消时认真清理。

如果继续迭代，我会优先把 SQLite 任务队列迁移到独立 worker，使用容器或沙箱增强恶意代码隔离，并为题目版本和翻译增加更完整的后台审核。当前版本适合课程本地实验和演示，不应未经加固直接暴露到公网。

最终结论

项目把课程 Step 1-6 和 Advance 连成了可演示、可追溯的主链路。完成不等于没有边界：数学题目质量仍需人工审查，恶意代码隔离仍需更强沙箱，平台相关结论必须以真实 Linux 和 CI 证据为准。

参考资料

- 课程 OJ 实验说明、Step 1-6、API、FAQ、评分标准与 Advance: <https://dbg-course.github.io/python-docs/oj/>
- FastAPI、Streamlit、SQLite、Python `asyncio` 与 Linux resource 官方文档
- 项目 `README.md`、`DEMO_GUIDE.md` 与 `PROJECT_WALKTHROUGH.md`